

(2) What is the object of your shipping expedition?

In Sentence 1, 'object' being tagged with the verb POS tag would be spoken with a different intonation as compared to its usage in Sentence 2, where it is being used as a noun in the query.

POS tagging is also useful in the important but challenging problem of word sense disambiguation. Polysemy of many languages means that a single word can have different meanings associated with it. POS tagging of the sentence can aid in word sense disambiguation. Consider the following two sentences:

(1) I bank with HSBC.

(2) I sat on the river bank.

We can use the POS tags to disambiguate between the different meanings associated with the word 'bank'.

Given the usefulness of POS tagging, now let us discuss the possible approaches to it. First, let us define the problem of POS tagging formally. Given a sentence consisting of words $\{w_1, w_2, w_3, \dots, w_n\}$ and a tag set $T = \{t_1, t_2, \dots, t_m\}$, the task is to identify the best possible tag sequence that can be assigned for the sentence S.

Just as in the case of various other tasks in NLP, POS tagging can also be performed using both supervised and unsupervised learning techniques. Supervised POS tagging requires that you are given a text Corpus C and a Tag set T. The Corpus C consists of a set of text documents and each document is a collection of sentences. Each Sentence S is a collection of words. Each word in S is tagged/labelled with the appropriate tag set element from T, resulting in a tag sequence $\langle t_1, t_2, \dots, t_n \rangle$ being associated with the Sentence S consisting of $\langle w_1, w_2, \dots, w_n \rangle$. Hence, we are given a set of $\langle \text{sentence, tag sequence corresponding to the sentence} \rangle$ as training data. The task is to learn a function that can predict/assign the tag sequence corresponding to the input test sentence, given the labelled training data.

While supervised POS tagging approaches perform well in terms of accuracy, the disadvantage is that they need the corpus to be labelled with the tags. Unsupervised POS tagging is based on finding equivalent word classes; however, without any labelled data, it cannot assign human readable tags to words. Hence, a small amount of manually labelled corpora can be used with unsupervised POS taggers to induce a mapping between human readable tags and the word equivalence classes induced by the tagger. We will discuss the unsupervised POS tagging in detail in the next column.

Another way of categorising the POS tagging approaches is as either rule based POS tagging techniques or stochastic POS tagging techniques. Rules can either be provided manually by linguistic

experts, or they can be automatically induced by learning from large text corpora which are tagged. The rules can then be used to tag an untagged sentence. Stochastic POS tagging techniques are based on finding the probabilities of the tag sequences associated with a given sentence. Assume that you have a black box, which can assign a probability of 'p[i]' to each possible tag sequence 'i' for sentence S. Given these probabilities, the stochastic approach can choose that tag sequence whose 'p[i]' is the maximum from among all possible tag sequences.

Now let us consider that we have a sentence S whose word sequence is $\langle w_1, w_2, \dots, w_n \rangle$. Each word 'w' can be tagged with one of the possible tags from tag set T. Let us assume that tag set T contains M tags. Hence, each word can be assigned one of M possible tags. Can you calculate how many tag sequences are possible for the given sentence S?

The number of possible tag sequences associated with S becomes very large and is equal to M^n . Hence, brute force search among all possible tag sequences is not efficient. Even if the probabilities are known for each possible tag sequence associated with S, it is computationally expensive to find that particular tag sequence whose probability is the highest.

Now can you think of ways by which we can avoid this brute force search? The question to our readers is this: Can you think of any solution from your algorithms course, that you have used in similar problems? For instance, consider the example of finding the shortest path from a source vertex to a destination vertex in a graph. Since there are exponentially many possible paths, a brute force listing of each path and its associated cost is computationally expensive. In such cases, can you recall what kind of algorithmic techniques you used to overcome this issue? A clue I would like to give you is to think about the dynamic programming approach. We will continue the discussion of POS tagging approaches in our next column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in the Xerox India Research Centre. Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group Computer Science Interview Training India at <http://www.linkedin.com/groups?home=HYPERLINK> <http://www.linkedin.com/groups?home=&gid=2339182> &HYPERLINK <http://www.linkedin.com/groups?home=&gid=2339182>